FML2ILV – Federated Machine Learning

Session 4: Federated Clustering

Assoc. Prof. Alexander Jung

10 April 2026 – IMC Krems

Glossary **Book** **GitHub**

Recap: The FL Design Workflow

Background: k-Means Clustering

Federated k-Means via GTVMin

Demo: Two-Device Federated k-Means

Summary

Table of Contents

Recap: The FL Design Workflow

Background: k-Means Clustering

Federated k-Means via GTVMin

Demo: Two-Device Federated k-Means

Summary

A Toy FL Network

- Each node i holds **local model parameters** $\mathbf{w}^{(i)} \in \mathbb{R}^d$
- Stack all local params into one vector:

$$\mathbf{w} = \left(\mathbf{w}^{(1)},\ \mathbf{w}^{(2)}\right) \in \mathbb{R}^{2d}$$

- Measure usefulness of $\mathbf{w}$ by local loss functions **plus** generalized total variation (GTV) penalty

The Three-Step FL Workflow

1. **Formulate** the learning task as GTVMin:

$$\min_{\mathbf{w}^{(1)},\ldots,\mathbf{w}^{(n)}} \sum_{i=1}^{n} L_i\left(\mathbf{w}^{(i)}\right) + \alpha \sum_{\{i,i'\}\in\mathcal{E}} A_{i,i'}\, d(\mathbf{w}^{(i)},\mathbf{w}^{(i')})$$

2. **Characterize** solutions via a fixed-point equation: $\widehat{\mathbf{w}} = \mathcal{F}(\widehat{\mathbf{w}})$

3. **Iterate** to obtain an FL algorithm: $\mathbf{w}^{(t+1)} = \mathcal{F}\big(\mathbf{w}^{(t)}\big)$

Today: apply this workflow to clustering problems.

Step 1: GTVMin with Euclidean Variation

Special case: squared Euclidean distance as variation measure and **differentiable** local loss functions:

$$\min_{\mathbf{w}^{(1)},\ldots,\mathbf{w}^{(n)}} \sum_{i=1}^{n} L_i\left(\mathbf{w}^{(i)}\right) + \alpha \sum_{\{i,i'\}\in\mathcal{E}} A_{i,i'} \left\|\mathbf{w}^{(i)} - \mathbf{w}^{(i')}\right\|_2^2$$

- First term: each device fits its local dataset
- Second term: penalizes disagreement between neighbours
- $\alpha > 0$: trade-off between local fit and global agreement

Step 2: Fixed-Point Characterization

GTVMin solution $\widehat{\mathbf{w}}^{(i)}$ satisfies the zero-gradient condition:

$$\nabla L_i\left(\widehat{\mathbf{w}}^{(i)}\right) \; + \; 2\alpha \sum_{i' \in \mathcal{N}^{(i)}} A_{i,i'} \left(\widehat{\mathbf{w}}^{(i)} - \widehat{\mathbf{w}}^{(i')}\right) \; = \; \mathbf{0}$$

Def. **block-wise operator** $\mathcal{F} = \left(\mathcal{F}^{(1)}, \ldots, \mathcal{F}^{(n)}\right)$,

$$\boxed{\begin{aligned} \mathcal{F}^{(i)}\left(\mathbf{w}^{(1)}, \ldots, \mathbf{w}^{(n)}\right) \; = \; & \mathbf{w}^{(i)} \; - \; \eta \nabla L_i\left(\mathbf{w}^{(i)}\right) \\ & - \; 2\eta\alpha \sum_{i' \in \mathcal{N}^{(i)}} A_{i,i'} \left(\mathbf{w}^{(i)} - \mathbf{w}^{(i')}\right) \end{aligned}}$$

- Zero-gradient $\Leftrightarrow$ fixed point: $\widehat{\mathbf{w}} = \mathcal{F}(\widehat{\mathbf{w}})$
- $\mathcal{F}^{(i)}$ uses local data $+$ neighbour params

Step 3: Fixed-Point Iteration $\Rightarrow$ FL Algorithm

Apply $\mathcal{F}$ repeatedly to obtain an iterative FL algorithm:

At each round t, every device i:

1. **Read** neighbour model parameters $\mathbf{w}^{(i',t)}, \ i' \in \mathcal{N}^{(i)}$
2. **Update** own model parameters:

$$\mathbf{w}^{(i,t+1)} = \mathcal{F}^{(i)}\big(\mathbf{w}^{(1,t)}, \ldots, \mathbf{w}^{(n,t)}\big)$$

3. **Share** updated $\mathbf{w}^{(i,t+1)}$ with neighbours

If $\mathcal{F}$ is a contractive operator, the iterates converge to the GTVMin solution $\widehat{\mathbf{w}} = \mathcal{F}(\widehat{\mathbf{w}})$.

Table of Contents

Recap: The FL Design Workflow

Background: k-Means Clustering

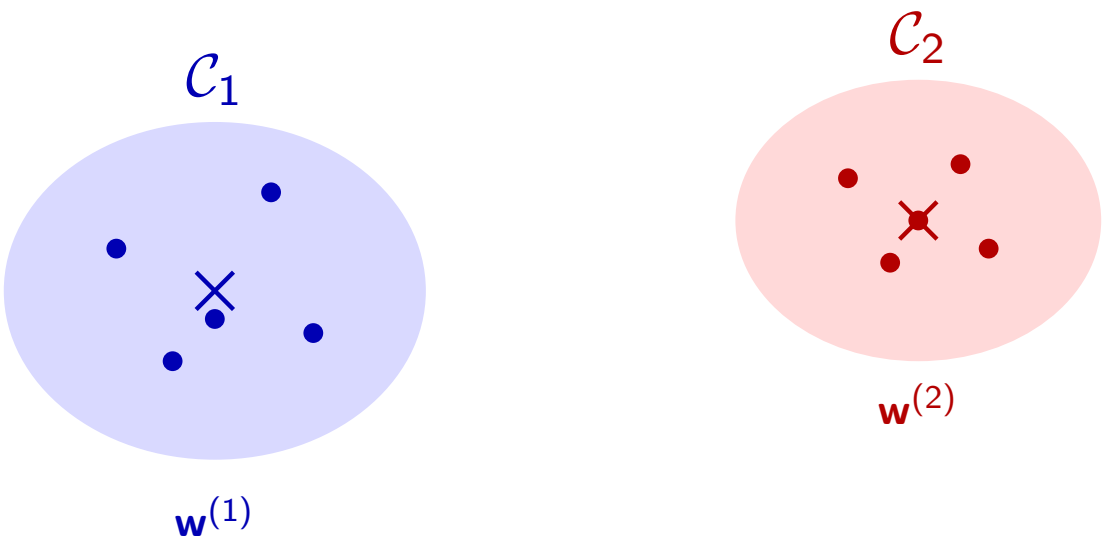
Federated k-Means via GTVMin

Demo: Two-Device Federated k-Means

Summary

The k-means Clustering Problem

Gven a dataset $\mathcal{D} = \{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}\} \subseteq \mathbb{R}^d$,



find cluster centroids $\mathbf{w}^{(1)}, \ldots, \mathbf{w}^{(k)}$ such that each data point $\mathbf{x}^{(r)}$ is maximally close to its nearest centroid, i.e., $\min_{c \in \{1,\ldots,k\}} \|\mathbf{x}^{(r)} - \mathbf{w}^{(c)}\|^2$ is small.

The k-means Optimization Problem

Given $\mathcal{D} = \{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(m)}\}$, stack all cluster centroids into one model parameter vector:

$$\mathbf{w} = \left(\mathbf{w}^{(1)}, \ldots, \mathbf{w}^{(k)}\right) \in \mathbb{R}^{d \cdot k}$$

$$\boxed{\min_{\mathbf{w}} \; L(\mathbf{w}) \quad \text{where} \quad L(\mathbf{w}) = \sum_{r=1}^{m} \min_{c \in \{1,\ldots,k\}} \|\mathbf{x}^{(r)} - \mathbf{w}^{(c)}\|^2}$$

- inner $\min_c$ assigns each point to nearest cluster centroid
- $L(\mathbf{w})$ is **not differentiable**
- $\Rightarrow$ no gradient descent

k-means: Lloyd's Algorithm

Despite non-differentiability, we can minimize $L(\mathbf{w})$ by alternating two steps:

1. **Assign** each point to its nearest cluster centroid (fix $\mathbf{w}$):

$$c^{(r)} = \arg\min_{c \in \{1,\ldots,k\}} \|\mathbf{x}^{(r)} - \mathbf{w}^{(c)}\|^2$$

2. **Update** each cluster centroid as the mean of its cluster (fix assignments):

$$\mathbf{w}^{(c)} = \frac{1}{|\mathcal{C}_c|} \sum_{\mathbf{x}^{(r)} \in \mathcal{C}_c} \mathbf{x}^{(r)}$$

Repeat until convergence. Each step decreases $L(\mathbf{w})$.

k-means in Practice

$$\min_{\mathbf{w}} \; L(\mathbf{w}) = \min_{\mathbf{w}} \; \sum_{r=1}^{m} \min_{c} \|\mathbf{x}^{(r)} - \mathbf{w}^{(c)}\|^2$$

```
km = KMeans(n_clusters=k).fit(X)
centroids = km.cluster_centers_
```

`KMeans.fit(X)` $\approx$ solve $\min_{\mathbf{w}} L(\mathbf{w})$ via Lloyd's algorithm (approximate, local minimum).

Table of Contents

Recap: The FL Design Workflow

Background: k-Means Clustering

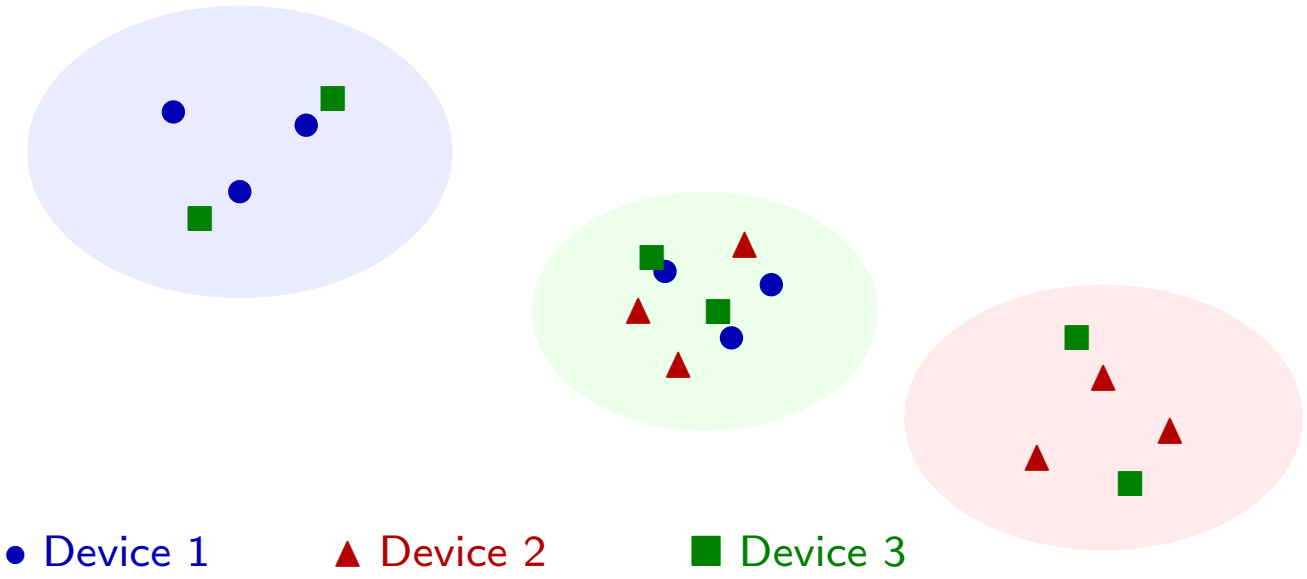
Federated k-Means via GTVMin

Demo: Two-Device Federated k-Means

Summary

Federated k-means: The Setting

n devices, each holding a private local dataset $\mathcal{D}^{(i)}$. The global dataset $\mathcal{D} = \mathcal{D}^{(1)} \cup \cdots \cup \mathcal{D}^{(n)}$ is **never pooled**.



- Each device sees only **partial view** of cluster structure
- Goal: learn cluster centroids without sharing raw data
- Only local cluster centroids are exchanged

Step 1: Formulate GTVMin for Federated k-means

n devices, each with local dataset $\mathcal{D}^{(i)}$ and local cluster centroids $\mathbf{w}^{(i)} = (\mathbf{w}^{(i,1)}, \ldots, \mathbf{w}^{(i,k)})$.

$$\min_{\{\mathbf{w}^{(i)}\}} \sum_{i=1}^{n} \underbrace{\sum_{\mathbf{x} \in \mathcal{D}^{(i)}} \min_{c} \left\| \mathbf{x} - \mathbf{w}^{(i,c)} \right\|_2^2}_{\text{local } k\text{-means loss } L_i\left(\mathbf{w}^{(i)}\right)} + \alpha \underbrace{\sum_{\{i,i'\} \in \mathcal{E}} d(\mathbf{w}^{(i)}, \mathbf{w}^{(i')})}_{\text{GTV penalty}}$$

- Local term: fit private data at each device
- GTV penalizes cluster centroid disagreement across edges
- How to choose $d(\mathbf{w}^{(i)}, \mathbf{w}^{(i')})$?

Discrepancy Measure for Cluster Centeroids

$$d(\mathbf{w}^{(i)}, \mathbf{w}^{(i')}) = \underbrace{\sum_{c=1}^{k} \min_{c'} \left\| \mathbf{w}^{(i,c)} - \mathbf{w}^{(i',c')} \right\|_2^2}_{\text{match } i\text{'s centroids to nearest in } i'} + \underbrace{\sum_{c=1}^{k} \min_{c'} \left\| \mathbf{w}^{(i',c)} - \mathbf{w}^{(i,c')} \right\|_2^2}_{\text{match } i'\text{'s centroids to nearest in } i}$$

- Symmetric: $d(\mathbf{w}^{(i)}, \mathbf{w}^{(i')}) = d(\mathbf{w}^{(i')}, \mathbf{w}^{(i)})$
- $d(\mathbf{w}^{(i)}, \mathbf{w}^{(i')}) = 0$ if and only if the set of cluster centroids is the same on both devices i and i'

Step 2: Block-Wise Fixed-Point Characterization

At convergence, each device i satisfies:

$$\boxed{\widehat{\mathbf{w}}^{(i)} = \underbrace{\arg\min_{\mathbf{w}^{(i)}} \left[L_i\left(\mathbf{w}^{(i)}\right) + \alpha \sum_{i' \in \mathcal{N}^{(i)}} d\big(\mathbf{w}^{(i)}, \widehat{\mathbf{w}}^{(i')}\big) \right]}_{\mathcal{F}^{(i)}\big(\widehat{\mathbf{w}}^{(1)}, \ldots, \widehat{\mathbf{w}}^{(n)}\big)}}$$

Block structure. Stack $\widehat{\mathbf{w}} = (\widehat{\mathbf{w}}^{(1)}, \ldots, \widehat{\mathbf{w}}^{(n)})$. The joint operator has one block per device:

$$\widehat{\mathbf{w}} = \mathcal{F}(\widehat{\mathbf{w}}) \quad \text{with} \quad \mathcal{F}(\mathbf{w}) = \big(\mathcal{F}^{(1)}(\mathbf{w}), \ldots, \mathcal{F}^{(n)}(\mathbf{w})\big)$$

device (block) i uses local $\mathcal{D}^{(i)}$ and neighbour params $\{\mathbf{w}^{(i')}\}_{i' \in \mathcal{N}^{(i)}}$

Step 3: Fixed-Point Iteration (FL Algorithm)

1. **Share:** node i sends centroids $\mathbf{w}^{(i,t)}$ to neighbours
2. **Update:** node i solves local subproblem

$$\mathbf{w}^{(i,t+1)} = \arg\min_{\mathbf{w}^{(i)}} \left[L_i\left(\mathbf{w}^{(i)}\right) + \alpha \sum_{i' \in \mathcal{N}^{(i)}} d(\mathbf{w}^{(i)}, \mathbf{w}^{(i',t)}) \right]$$

3. **Repeat** until convergence

Step 3: The Local Subproblem (Expanded)

$$\mathbf{w}^{(i,t+1)} = \underset{\mathbf{w}^{(i,1)},\ldots,\mathbf{w}^{(i,k)}}{\arg\min} \underbrace{\sum_{r=1}^{m_i} \min_c \left\| \mathbf{x}^{(r)} - \mathbf{w}^{(i,c)} \right\|_2^2}_{\text{local } k\text{-means loss}}$$

$$+ \alpha \sum_{i' \in \mathcal{N}^{(i)}} A_{i,i'} \Bigg(\underbrace{\sum_{c'=1}^{k} \min_c \left\| \mathbf{w}^{(i',c',t)} - \mathbf{w}^{(i,c)} \right\|_2^2}_{\text{assign neighbour centroids to nearest own } (k\text{-means-like!})}$$

$$+ \underbrace{\sum_{c=1}^{k} \min_{c'} \left\| \mathbf{w}^{(i,c)} - \mathbf{w}^{(i',c',t)} \right\|_2^2}_{\text{assign own centroids to nearest neighbour (dropped in approx.)}} \Bigg)$$

Almost looks like k-means on an augmented dataset (original points + neighbour centroids).

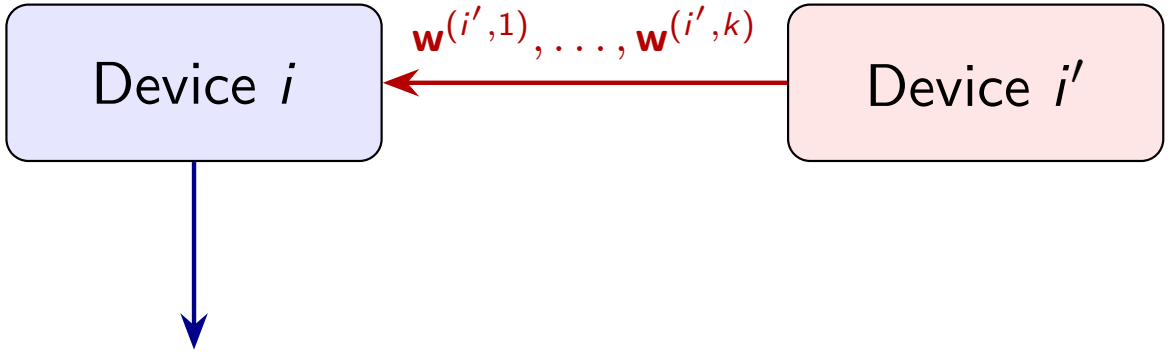
Approximation

$$\mathbf{w}^{(i,t+1)} \approx \underset{\mathbf{w}^{(i,1)},\ldots,\mathbf{w}^{(i,k)}}{\arg\min} \underbrace{\sum_{r=1}^{m_i} \min_c \left\| \mathbf{x}^{(r)} - \mathbf{w}^{(i,c)} \right\|_2^2}_{\text{local data}}$$

$$+ \sum_{i' \in \mathcal{N}^{(i)}} \underbrace{R_{i'} \sum_{c'=1}^{k} \min_c \left\| \mathbf{w}^{(i',c',t)} - \mathbf{w}^{(i,c)} \right\|_2^2}_{R_{i'} \text{ copies of each neighbour centroid as pseudo-data}}$$

this is k-**means on an augmented dataset** where each neighbour centroid $\mathbf{w}^{(i',c',t)}$ is repeated $R_{i'} = \alpha\, A_{i,i'}$ times:

Augmented Dataset: Illustration



$$\mathcal{D}^{(i,\mathrm{aug})} = \mathcal{D}^{(i)} \cup \{\underbrace{\mathbf{w}^{(i',c')}, \ldots, \mathbf{w}^{(i',c')}}_{R_{i'} \text{ copies}}\}_{c'=1}^{k}$$

run k-means on $\mathcal{D}^{(i,\mathrm{aug})}$

- no raw data exchanged; only cluster centroids
- neighbour cluster centroids act as pseudo-data
- implementable with `sklearn.cluster.KMeans`

Beyond Hard Clustering: Federated GMM

The data augmentation idea generalizes to **soft clustering**:

- use neg. log-likelihood of **GMM** as $L(\mathbf{w})$
- use **symmetrized KL divergence** as $d(\mathbf{w}^{(i)}, \mathbf{w}^{(i')})$
- approximate GTVMin via local GMM fits on augmented data
- augmentation: draw **samples from neighbour GMMs** (not raw data)

Table of Contents

Recap: The FL Design Workflow

Background: k-Means Clustering

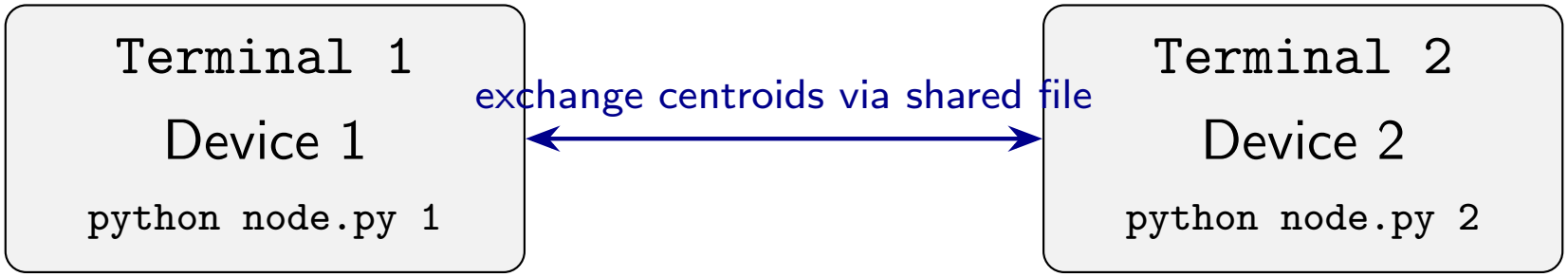
Federated k-Means via GTVMin

Demo: Two-Device Federated k-Means

Summary

Demo Setup

Two terminal windows, each running a Python script that simulates one FL device.



- each device generates local data (2D Gaussian blobs)
- communication via shared JSON file
- watch centroids converge across iterations

Demo: What to Observe

1. **Iteration 0:** local k-means only — centroids may disagree
2. **Iteration 1+:** augment with neighbour centroids — start to align
3. **Convergence:** both devices agree on centroids despite different data

Open two terminals and run:
`python demos/Session4_node.py 1` and `python demos/Session4_node.py 2`

Demo: The Update Step Explained

At each round, device i performs the following update:

1. **Receive** cluster centroids from each neighbour $i' \in \mathcal{N}^{(i)}$
2. **Augment** local dataset with $R_{i'}$ copies of each received cluster centroid:

$$\mathcal{D}^{(i,\mathrm{aug})} = \mathcal{D}^{(i)} \cup \bigcup_{i' \in \mathcal{N}^{(i)}} \{\underbrace{\mathbf{w}^{(i',c)}, \ldots, \mathbf{w}^{(i',c)}}_{R_{i'} \text{ copies}} \mid c = 1, \ldots, k\}$$

3. **Re-run** standard k-means on $\mathcal{D}^{(i,\mathrm{aug})}$ $\rightarrow$ updated cluster centroids

Demo: Core Algorithm (Pseudocode)

```
# At each FL iteration:

# 1. SHARE: write own centroids to shared
   file
write_centroids(node_id, centroids)

# 2. READ: load neighbour's centroids
nb_centroids = read_centroids(neighbour_id)

# 3. UPDATE: augment + re-cluster
#    R = alpha * A_edge matches GTVMin
   coupling
R = int(ALPHA * A_EDGE)
X_aug = np.vstack([X_local] + [nb_centroids]
    * R)
kmeans = KMeans(n_clusters=K).fit(X_aug)
centroids = kmeans.cluster_centers_
```

Table of Contents

Recap: The FL Design Workflow

Background: k-Means Clustering

Federated k-Means via GTVMin

Demo: Two-Device Federated k-Means

Summary

Summary

- **Federated k-means:** couple local problems via GTV on centroids
- **Solver:** approximate fixed-point iter. via k-means on augmented data (local data + neighbour centroids as pseudo-data)
- **Federated GMM:** same idea, exchange GMM samples instead of cluster centroids
- **Demo:** two terminals, centroids via shared file or socket

What's Next?

Tuesday, 14 April: Robustness, Cyber-Security & Privacy in FL.